

6-DAY INSTRUCTOR-LED TECHNICAL TRAINING

Core Python with AI-Integrated Development

A fast-track, hands-on bootcamp for experienced developers: master Python's object model, data structures, and standard library — while working AI coding assistants into every stage of the development lifecycle.

- Prior dev experience required
- Python 3.14 · VS Code
- AI Coding Assistant hands-on labs

```
day_01.py

developer =
Engineer(role="builder")

assistant =
configure_ai_assistant()

for day in course.days:
    day.learn()
    day.build_with(assistant)
    day.ship()
```

6-Day Agenda at a Glance

D1

Day 1

Conceptual Foundations, AI Coding Assistant Setup & Python/String Fundamentals

D2

Day 2

AI Coding Agents, Control Flow, Iterators & Core Data Structures (Lists, Tuples, Sets)

D3

Day 3

Dictionaries, Nested & Extended Data Structures, and File Handling

D4

Day 4

Modules & Packages, OOP, Comprehensions, and Exceptions/Debugging/AI-Assisted Testing

D5

Day 5

Core Libraries, Automation, and AI Tools for Developers (Copilot, Context Engineering & Adoption)

D6

Day 6

Capstone Project + Evaluation

DAY 5

Core Libraries, Automation, and AI Tools for Developers (Copilot, Context Engineering & Adoption)

- Core Standard Libraries: os, sys, re
- Parsing File Formats: CSV, XLSX, XML, HTML & Intro to pandas/numpy
- Automation in Practice: Process, Workflow, Network & Web
- GitHub Copilot, Context Engineering & Adoption
- Dates, Time & Data Persistence
- Concurrency Basics, CLI Input & Virtual Environments
- AI Tools for Developers

What You'll Be Able To Do

1

Use `os`, `sys`, and `re` confidently to script filesystem, environment, and text-pattern tasks.

2

Persist and exchange data with `datetime/time`, `json`, `pickle`, `shelve`, `csv`, `xlsx`, and `xml`, and take the first practical steps into `pandas/numpy`.

3

Explain when to reach for `threading`, `multiprocessing`, or `asyncio`, and isolate project dependencies with `venv`.

4

Write short automation scripts for file, workflow, network, and web tasks using the standard library and `requests`.

5

Apply AI assistants to optimize, generate, and debug Python code while independently validating every claim they make.

6

Use context engineering and GitHub Copilot effectively, and describe a realistic path for adopting AI coding tools on a team.

Core Standard Libraries: os, sys, re

os, sys, and re are built-in modules for filesystem/OS interaction, interpreter introspection, and text pattern matching — no installs required.

THE PROBLEM IT SOLVES

Real scripts must touch the filesystem, read runtime/CLI context, and validate or extract structured text.

WHY IT MATTERS

These three modules show up in almost every non-trivial Python script you will write or maintain.

WHERE IT'S USED

Build scripts, log analyzers, deployment tooling, config loaders, and CI/CD pipelines all depend on them.

REAL-WORLD ANALOGY

"os/sys are your OS and runtime control panel; re is a specialized search-and-replace engine for text."

os and sys Essentials

OS MODULE

- `os.getcwd()` / `os.chdir()` — inspect or change the working directory
- `os.listdir(path)`, `os.path.join()` — list and build paths portably
- `os.environ` — mapping-like access to environment variables
- `os.makedirs(path, exist_ok=True)` — create nested dirs safely

SYS MODULE

- `sys.argv` — list of raw command-line arguments (`argv[0]` is the script name)
- `sys.path` — list of directories Python searches for imports
- `sys.exit(code)` — stop the process with an exit status
- `sys.stdin` / `stdout` / `stderr` — the standard I/O stream objects

DEVELOPER PERSPECTIVE

- Prefer `os.path.join()`/`pathlib` over manual string concatenation for portability
- `sys.exit(1)` signals failure to shells/CI; 0 (or omitted) signals success

os / sys Syntax

```
syntax.py

import os
import sys

cwd = os.getcwd()
entries = os.listdir(cwd)
os.makedirs("reports/2026", exist_ok=True)

print(sys.argv)      # ['script.py', 'input.txt']
print(sys.platform)  # 'linux', 'win32', 'darwin'
if not entries:
    sys.exit(1)       # nonzero = failure, for shells/CI
```

HOW IT READS

- `os.getcwd()/os.listdir()` are the read side of filesystem inspection
- `os.makedirs(..., exist_ok=True)` avoids a `FileExistsError` on reruns
- `sys.argv[0]` is always the script path — real args start at index 1
- `sys.platform` lets one script branch behavior across OSes
- `sys.exit(code)` raises `SystemExit`; code 0 means success by convention

Basic Example

```
example.py

import os
import sys

target_dir = sys.argv[1] if len(sys.argv) > 1 else "."

py_files = [f for f in os.listdir(target_dir) if f.endswith(".py")]

print(f"Found {len(py_files)} Python files in {target_dir}")
sys.exit(0 if py_files else 1)
```

WHAT'S HAPPENING

- `sys.argv[1]` reads an optional directory argument, defaulting to the current one
- A list comprehension filters directory entries by extension
- The exit code communicates success/failure to a calling shell script or CI job

CONSOLE OUTPUT

```
Found 3 Python files in .
```

re Module: Matching Text

KEY TERMINOLOGY

- pattern — a regex string, usually a raw string `r"..."`
- match object — returned on success; `None` on failure
- `group(n)` — captured text from the `n`th parenthesized group
- greedy vs lazy — `'.*'` takes as much as possible, `'.*?'` as little

CORE FUNCTIONS

- `re.match(pattern, s)` — matches only at the start of the string
- `re.search(pattern, s)` — finds the first match anywhere
- `re.findall(pattern, s)` — returns all non-overlapping matches as a list
- `re.sub(pattern, repl, s)` — replaces all matches

DEVELOPER PERSPECTIVE

- Precompile with `re.compile()` when reusing a pattern across many strings
- Always use raw strings (`r"\d+"`) — plain strings mangle backslash escapes

re Syntax: Groups & Substitution

```
syntax.py

import re

pattern =
re.compile(r"(?P<level>ERROR|WARN) \s+ (?P<code>\d{3}) ")

m = pattern.search("WARN 404 not found")
print(m.group("level"))    # 'WARN'
print(m.group("code"))    # '404'

clean = re.sub(r"\s+", " ", "too many spaces")
print(clean)               # 'too many spaces'
```

HOW IT READS

- `(?P<name>...)` defines a named capturing group, read via `.group("name")`
- `search()` scans the whole string for the first match, unlike `match()`
- `re.sub` replaces every match of the pattern with the replacement string
- `\s+` matches one-or-more whitespace characters — common for cleanup
- Compiling once with `re.compile()` avoids re-parsing the pattern each call

Practical Example

```
practical_example.py

import re

log_lines = [
    "2026-09-12 10:03:11 ERROR db connection lost",
    "2026-09-12 10:03:15 INFO retrying connection",
    "2026-09-12 10:03:20 ERROR timeout after 5s",
]

pattern = re.compile(
    r"(?P<ts>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}) "
    r"(?P<level>\w+) (?P<msg>.+)"
)

errors = []
for line in log_lines:
    m = pattern.match(line)
    if m and m.group("level") == "ERROR":
        errors.append((m.group("ts"), m.group("msg")))
```

WHAT'S HAPPENING

- Named groups extract timestamp, level, and message from each log line
- `match()` is appropriate here since every line has the same fixed prefix
- Filtering on `m.group("level")` isolates only ERROR entries
- Extracted tuples make downstream reporting (CSV, alerting) straightforward

CONSOLE OUTPUT

```
2026-09-12 10:03:11 -> db connection lost
2026-09-12 10:03:20 -> timeout after 5s
```

Common Mistakes

✗ INCORRECT

```
path = dir + "/" + filename
```

✓ CORRECT

```
path = os.path.join(dir, filename)
```

✗ INCORRECT

```
pattern = "\d+"
```

✓ CORRECT

```
pattern = r"\d+"
```

✗ INCORRECT

```
re.match(r"\d+", "id: 42")
```

✓ CORRECT

```
re.search(r"\d+", "id: 42")
```

✗ INCORRECT

```
if os.listdir(path):  
    ...
```

✓ CORRECT

```
if os.path.isdir(path) and os.listdir(path):
```

✗ INCORRECT

```
sys.exit("done")
```

✓ CORRECT

```
print("done")  
sys.exit(0)
```

Common Mistakes — Why It Matters

- Mistake 1: Manual concatenation breaks on Windows and with trailing/missing slashes.
- Mistake 2: Non-raw strings can mangle backslash escape sequences before `re` even sees them.
- Mistake 3: `match()` only anchors at index 0, so it misses matches later in the string.
- Mistake 4: `listdir()` raises `FileNotFoundError/NotADirectoryError` if path is invalid.
- Mistake 5: `sys.exit(str)` prints the string to `stderr` and exits with code 1 — usually not intended.

Best Practices

PEP 8 & READABILITY

- Prefer `pathlib.Path` for new code; `os.path` is fine for legacy/porting work
- Give compiled regex patterns descriptive names, not just ``pattern``

MAINTAINABILITY

- Keep regexes simple and commented (`re.VERBOSE`) once they exceed ~40 characters
- Wrap OS calls that can fail (missing file, permissions) in explicit checks or `try/except`

PERFORMANCE

- Compile a regex once with `re.compile()` if it runs inside a loop
- Avoid catastrophic backtracking: bound repetitions instead of nested unbounded quantifiers

AI-Assisted Development

PROMPT

Review this log-parsing function for correctness and suggest a more robust regex, including edge cases I might be missing.

```
before_ai.py

def get_errors(lines):
    out = []
    for l in lines:
        if "ERROR" in l:
            out.append(l.split(" ")[2])
    return out
```

AI ASSISTANT SUGGESTS

- Suggests replacing brittle string splitting with a named-group regex
- Flags that `"ERROR"` in `l` also matches messages that merely mention the word
- Recommends anchoring the level field to a fixed position via the pattern
- Points out missing handling for lines that don't match the expected format

DEVELOPER MUST STILL VERIFY

- Run the suggested regex against real, messy log samples, not just the happy path
- Confirm named groups exist and are spelled correctly before trusting `.group()` calls
- Check performance on large log files, not just a 3-line demo

Hands-on Exercise

PROBLEM STATEMENT

Write a script that scans a directory (via `sys.argv`) for `.log` files, and for each file uses `re` to extract and count occurrences of `ERROR`, `WARN`, and `INFO` lines.

INPUT

A directory containing `app.log` with mixed `ERROR`/`WARN`/`INFO` lines

EXPECTED OUTPUT

```
app.log -> {'ERROR': 2, 'WARN': 1, 'INFO': 5}
```

HINTS (no full solution here — give it a try first!)

- Use `os.listdir()` plus a filename filter, or glob for `*.log`
- One compiled pattern with a named 'level' group can serve every file
- `collections.Counter` is a clean way to tally the extracted levels

Dates, Time & Data Persistence

datetime/time model calendar dates and durations; pickle/json/shelve save Python data between program runs.

THE PROBLEM IT SOLVES

Programs need to reason about timestamps and durations, and must survive restarts without losing state.

WHY IT MATTERS

Almost every service logs timestamps, schedules work, or caches/persists data to disk or a store.

WHERE IT'S USED

Scheduling jobs, expiring caches, audit logs, config files, and simple local data stores all rely on these modules.

REAL-WORLD ANALOGY

"datetime is your calendar and stopwatch; json/pickle/shelve are filing cabinets for Python data."

datetime & time

KEY TYPES

- `datetime.date` — calendar date (year, month, day)
- `datetime.datetime` — date + time of day, to microsecond precision
- `datetime.timedelta` — a duration; supports +/- arithmetic on dates
- `time.time()` — seconds since the epoch (float, for measuring elapsed time)

FORMATTING & PARSING

- `strftime(fmt)` — datetime object -> formatted string
- `strptime(s, fmt)` — formatted string -> datetime object
- Common codes: %Y %m %d %H %M %S

DEVELOPER PERSPECTIVE

- Use `datetime.now()` for wall-clock time, `time.time()` for elapsed/perf timing
- Store/compare datetimes in UTC internally; convert to local time only for display

Formatting & Parsing Dates

```
syntax.py

from datetime import datetime, timedelta
import time

now = datetime.now()
formatted = now.strftime("%Y-%m-%d %H:%M:%S")

parsed = datetime.strptime("2026-09-13", "%Y-%m-%d")
deadline = parsed + timedelta(days=7)

start = time.time()
# ... work happens here ...
elapsed = time.time() - start
```

HOW IT READS

- `strftime` turns a `datetime` into a display-ready string using format codes
- `strptime` is the inverse: parse a string into a `datetime` object
- `timedelta` supports arithmetic directly on `date`/`datetime` objects
- `time.time()` returns a float epoch timestamp, ideal for elapsed-time measurement
- `time.sleep(seconds)` pauses execution — useful for polling loops and demos

Basic Example

```
example.py

from datetime import datetime

def days_until(deadline_str):
    deadline = datetime.strptime(deadline_str, "%Y-%m-%d")
    remaining = deadline - datetime.now()
    return max(remaining.days, 0)

print(days_until("2026-12-31"))
```

WHAT'S HAPPENING

- `strptime` parses the string deadline into a real datetime object
- Subtracting two datetimes yields a `timedelta` with a `.days` attribute
- `max(..., 0)` guards against reporting negative days for a past deadline

CONSOLE OUTPUT

```
109
```

Persisting Data: pickle, json, shelve

JSON

- Text-based, language-agnostic — safe to share across systems/services
- `json.dump(obj, file)` / `json.load(file)`; dumps/loads for strings
- Only supports basic types: dict, list, str, int, float, bool, None

PICKLE

- Binary, Python-specific — can serialize almost any Python object
- `pickle.dump(obj, file)` / `pickle.load(file)`
- Never unpickle data from an untrusted source — it can execute arbitrary code

SHELVE (BRIEF)

- A persistent, dict-like store backed by a file (built on pickle internally)
- `shelve.open("data")` gives a dict you can read/write; call `.close()` when done

json & pickle Syntax

```
syntax.py

import json
import pickle

config = {"retries": 3, "timeout": 30}

with open("config.json", "w") as f:
    json.dump(config, f, indent=2)

with open("config.json") as f:
    loaded = json.load(f)

with open("state.pkl", "wb") as f:
    pickle.dump(config, f)

with open("state.pkl", "rb") as f:
    restored = pickle.load(f)
```

HOW IT READS

- `json.dump/load` work on text files; `indent=2` makes output human-readable
- `pickle` uses binary mode ('wb'/'rb') and preserves richer Python types
- `json` is the right default for config/interop; `pickle` for internal Python-only caches
- Both round-trip data losslessly for the types they support

Practical Example

```
practical_example.py

import json
import shelve

def save_settings(path, settings):
    with open(path, "w") as f:
        json.dump(settings, f, indent=2)

def load_settings(path, defaults):
    try:
        with open(path) as f:
            return json.load(f)
    except FileNotFoundError:
        return defaults

settings = load_settings("app.json", {"theme": "dark"})
save_settings("app.json", settings)

with shelve.open("cache") as db:
```

WHAT'S HAPPENING

- `load_settings` falls back to defaults on first run, when no file exists yet
- `json` is used for the human-editable settings file
- `shelve` behaves like a dict but persists automatically to disk
- The ``with`` context manager ensures files/shelves are properly closed

CONSOLE OUTPUT

2026-09-13

Common Mistakes

✗ INCORRECT

```
pickle.load(open(url_download, "rb"))
```

✓ CORRECT

```
# only unpickle files you produced/trust
```

✗ INCORRECT

```
json.dump(obj, f) # obj has a datetime
```

✓ CORRECT

```
json.dump(obj, f, default=str)
```

✗ INCORRECT

```
d = datetime.strptime(s, "%Y-%d-%m")
```

✓ CORRECT

```
d = datetime.strptime(s, "%Y-%m-%d")
```

✗ INCORRECT

```
shelve.open("cache")  
db["x"] = 1
```

✓ CORRECT

```
with shelve.open("cache") as db:  
    db["x"] = 1
```

Common Mistakes — Why It Matters

- Mistake 1: Unpickling untrusted data can execute arbitrary code on your machine.
- Mistake 2: json can't serialize datetime objects by default and raises TypeError.
- Mistake 3: Mixed-up format codes silently swap month/day for many valid inputs.
- Mistake 4: Forgetting to close a shelve can leave writes unflushed to disk.

Best Practices

PEP 8 & READABILITY

- Store datetimes in ISO 8601 strings ("%Y-%m-%dT%H:%M:%S") in JSON for portability

MAINTAINABILITY

- Prefer json for anything crossing a process/service/language boundary
- Reserve pickle for same-codebase caches (e.g. memoized results) only

SECURITY

- Never call pickle.load on data from the network or an unauthenticated source
- Validate/whitelist fields after json.load — don't assume the shape is correct

AI-Assisted Development

PROMPT

This function saves app state with pickle. Is this safe, and should I use something else? Suggest a safer alternative if needed.

```
before_ai.py

def save_state(state, path):
    with open(path, "wb") as f:
        pickle.dump(state, f)

def load_state(path):
    with open(path, "rb") as f:
        return pickle.load(f)
```

AI ASSISTANT SUGGESTS

- Flags that pickle.load is unsafe if the file could ever come from outside the app
- Suggests switching to json if `state` only contains basic types
- Recommends validating the loaded structure before using it
- Notes that pickle format can break across incompatible Python/library versions

DEVELOPER MUST STILL VERIFY

- Confirm no part of the pickle file path is ever attacker-influenced
- Check that all values in `state` are actually JSON-serializable before switching
- Test loading state saved by an older version of the app

Hands-on Exercise

PROBLEM STATEMENT

Write a small script that logs each run's start time to a JSON file (list of ISO timestamps), keeping only the last 5 runs, and prints how many seconds elapsed since the previous run.

INPUT

`history.json` containing a JSON list of previous ISO timestamps (or missing on first run)

EXPECTED OUTPUT

```
Seconds since last run: 86423.5
History now has 5 entries.
```

HINTS (no full solution here — give it a try first!)

- Use `datetime.now().isoformat()` to generate each entry
- Load with a `try/except FileNotFoundError` for the first-ever run
- Slice the list to the last 5 entries with `history[-5:]` before saving

Parsing File Formats & Intro to pandas/numpy

Python ships parsers for the most common exchange formats (csv, xml, basic html) and has a rich ecosystem (openpyxl, pandas, numpy) for the rest.

THE PROBLEM IT SOLVES

Real integrations receive data as CSV exports, spreadsheets, XML feeds, or scraped HTML — not neat Python objects.

WHY IT MATTERS

Nearly every data-integration or reporting task starts with reading one of these formats correctly.

WHERE IT'S USED

ETL scripts, report generators, vendor data feeds, web scraping, and analytics notebooks.

REAL-WORLD ANALOGY

"Each format is a different shipping container; the parser is the tool that knows how to unpack that specific container."

csv Module

KEY TERMINOLOGY

- `csv.reader` — iterates rows as plain lists of strings
- `csv.DictReader` — iterates rows as dicts keyed by header
- `csv.writer` / `DictWriter` — the corresponding write-side API
- `dialect` — delimiter, quoting, and line-terminator conventions

HOW IT WORKS

- Open the file, wrap it in `csv.reader()/DictReader()`, then iterate
- Every field comes back as a `str` — you convert types yourself

DEVELOPER PERSPECTIVE

- Always open CSV files with `newline=""` to avoid extra blank rows on Windows
- `DictReader` is usually worth the small overhead for self-documenting code

csv Syntax

```
syntax.py

import csv

with open("orders.csv", newline="") as f:
    reader = csv.DictReader(f)
    rows = [row for row in reader]

with open("summary.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["id", "total"])
    writer.writeheader()
    writer.writerow({"id": "1", "total": "42.50"})
```

HOW IT READS

- DictReader turns each row into a dict using the header row as keys
- `newline=""` is the documented-correct way to open CSV files in Python
- DictWriter needs fieldnames up front and `writeheader()` to emit the header row
- All values are strings — cast to `int`/`float`/`Decimal` explicitly as needed

Basic Example

```
example.py

import csv

total = 0.0
with open("orders.csv", newline="") as f:
    for row in csv.DictReader(f):
        total += float(row["amount"])

print(f"Total: {total:.2f}")
```

WHAT'S HAPPENING

- DictReader lets us access columns by name ("amount") instead of position
- Each value must be explicitly cast to float before arithmetic
- f-string formatting (:.2f) controls decimal display

CONSOLE OUTPUT

```
Total: 1284.50
```

xlsx, xml, and html

XLSX (OPENPYXL)

- `openpyxl.load_workbook(path)` opens a workbook; `wb.active` or `wb["Sheet1"]` a sheet
- `ws.iter_rows(values_only=True)` yields row tuples of cell values
- `pandas.read_excel()` is the faster path once you're doing analysis, not just I/O

XML (XML.ETREE.ELEMENTTREE)

- `ET.parse(path).getroot()` returns the root Element
- `element.findall("tag")` / `element.find("tag")` navigate children
- `element.attrib` is a dict of attributes; `element.text` is inner text

HTML (ECOSYSTEM NOTE)

- `html.parser.HTMLParser` gives low-level, callback-based parsing in the stdlib
- BeautifulSoup (3rd-party) is the de-facto standard for robust, forgiving HTML parsing

xml.etree.ElementTree Syntax

```
syntax.py

import xml.etree.ElementTree as ET

xml_data = """
<orders>
  <order id="1"><item>Widget</item><qty>4</qty></order>
  <order id="2"><item>Gadget</item><qty>2</qty></order>
</orders>
"""

root = ET.fromstring(xml_data)
for order in root.findall("order"):
    print(order.attrib["id"], order.find("item").text,
          order.find("qty").text)
```

HOW IT READS

- ET.fromstring parses an XML string directly (ET.parse reads from a file)
- findall("order") returns every matching child Element as a list
- attrib exposes tag attributes as a plain dict
- find("item").text reads the text content of a nested element

Practical Example

```
practical_example.py

import xml.etree.ElementTree as ET

def parse_orders(path):
    root = ET.parse(path).getroot()
    orders = []
    for order in root.findall("order"):
        orders.append({
            "id": order.attrib["id"],
            "item": order.find("item").text,
            "qty": int(order.find("qty").text),
        })
    return orders

for o in parse_orders("orders.xml"):
    print(o)
```

WHAT'S HAPPENING

- ET.parse(path) reads directly from a file on disk, unlike fromstring
- Each order Element is converted into a plain dict for easier downstream use
- int() conversion turns the text quantity into a usable number
- The same pattern generalizes to any well-formed XML feed

CONSOLE OUTPUT

```
{'id': '1', 'item': 'Widget', 'qty': 4}
{'id': '2', 'item': 'Gadget', 'qty': 2}
```

pandas & numpy: What's Next After Core Python

WHY THEY MATTER

- `pandas.DataFrame` replaces list-of-dicts for tabular data with vectorized operations
- `numpy.ndarray` replaces list-of-numbers with fast, memory-efficient array math
- Both are separate installs (`pip install pandas numpy`) — not `stdlib`

CORE OPERATIONS

- `pd.read_csv(path)` loads a CSV straight into a `DataFrame`
- `df["col"]`, `df.loc[...]`, `df[df.col > x]` — selection and filtering
- `np.array([...]) + / *` operate element-wise, no explicit loop needed

DEVELOPER PERSPECTIVE

- Reach for pandas once you're filtering/aggregating/joining tabular data repeatedly
- A numpy vectorized op is typically 10-100x faster than the equivalent Python loop

Advanced / Developer Example

```
advanced_example.py

import pandas as pd
import numpy as np

df = pd.read_csv("orders.csv")
big_orders = df[df["amount"] > 100]
print(big_orders[["id", "amount"]].head())

amounts = np.array(df["amount"])
print("Mean:", amounts.mean())
print("Discounted:", amounts * 0.9)
```

WHAT'S HAPPENING

- `pd.read_csv` replaces manual `csv.DictReader` loops for tabular analysis
- Boolean indexing (`df["amount"] > 100`) filters rows without an explicit loop
- `np.array(...)` converts a pandas Series/list into a vectorized numpy array
- `amounts * 0.9` applies the discount to every element in one vectorized call

CONSOLE OUTPUT

```
Mean: 128.45
Discounted: [ 45.  108.9 ...]
```

Common Mistakes

✗ INCORRECT

```
open("data.csv")
```

✓ CORRECT

```
open("data.csv", newline="")
```

✗ INCORRECT

```
order.find("qty").text + 1
```

✓ CORRECT

```
int(order.find("qty").text) + 1
```

✗ INCORRECT

```
for row in df: print(row)
```

✓ CORRECT

```
for _, row in df.iterrows(): print(row)
```

✗ INCORRECT

```
amounts = [1, 2, 3]  
total = amounts * 2
```

✓ CORRECT

```
amounts = np.array([1, 2, 3])  
total = amounts * 2
```

Common Mistakes — Why It Matters

- Mistake 1: Omitting `newline=""` can introduce spurious blank rows with the `csv` module on Windows.
- Mistake 2: `ElementTree` text is always a string; arithmetic needs an explicit cast.
- Mistake 3: Iterating a `DataFrame` directly yields column names, not rows.
- Mistake 4: Multiplying a Python list by 2 repeats it; only a numpy array multiplies element-wise.

Best Practices

MAINTAINABILITY

- Validate expected columns/tags exist before processing (fail fast with a clear error)
- Isolate parsing logic in its own function so the data source can change later

PERFORMANCE

- Prefer pandas/numpy vectorized operations over row-by-row Python loops on large data

SECURITY

- Disable external entity resolution or use defusedxml when parsing XML from untrusted sources

AI-Assisted Development

PROMPT

Convert this manual CSV-parsing loop into a pandas-based version, and tell me what edge cases pandas handles that my loop doesn't.

before_ai.py

```
import csv

total = 0
with open("orders.csv") as f:
    for row in csv.DictReader(f):
        total += float(row["amount"])
print(total)
```

AI ASSISTANT SUGGESTS

- Proposes `pd.read_csv(...)[\"amount\"].sum()` as a one-line equivalent
- Notes pandas handles missing values (NaN) more gracefully than a raw `float()` cast
- Warns that dtype inference can silently misread a numeric-looking column as text
- Suggests specifying dtype explicitly for money/ID columns

DEVELOPER MUST STILL VERIFY

- Compare the pandas total against the original loop's total on the same file
- Check how each approach behaves on a row with a blank or malformed amount
- Confirm the pandas dependency is acceptable for this project's deployment target

Hands-on Exercise

PROBLEM STATEMENT

Read products.csv into a pandas DataFrame, compute total inventory value (price * quantity) per category, and also parse a small catalog.xml file to cross-check that every XML product id exists in the CSV.

INPUT

```
products.csv (id, category, price, quantity) and catalog.xml  
(list of <product id=...>)
```

EXPECTED OUTPUT

```
Electronics: 4820.00  
Home: 1390.50  
All XML ids found in CSV.
```

HINTS (no full solution here — give it a try first!)

- Use `df.groupby("category").apply(...)` or a computed column then `groupby().sum()`
- Build a set of CSV ids and check XML ids with ``not in`` for the cross-check
- `int()/float()` cast XML text fields before doing arithmetic on them

Concurrency Basics, CLI Input & Virtual Environments

threading/multiprocessing/asyncio give you three different concurrency models; argparse and venv are everyday tooling for scripts and dependency isolation.

THE PROBLEM IT SOLVES

Scripts often need to do more than one thing at once, accept structured CLI input, and avoid dependency conflicts between projects.

WHY IT MATTERS

Choosing the wrong concurrency model wastes effort; skipping venvs causes 'works on my machine' bugs.

WHERE IT'S USED

Automation scripts, CLI tools, web scrapers, and any multi-project development machine.

REAL-WORLD ANALOGY

"Threads/processes/async are different ways to get multiple errands done; a venv is a separate toolbox per project."

threading, multiprocessing, asyncio — Conceptual Overview

THREADING

- Multiple threads share one process's memory; good for I/O-bound waiting
- Python's GIL means only one thread runs Python bytecode at a time

MULTIPROCESSING

- Separate processes, separate memory — true parallelism, bypasses the GIL
- Best for CPU-bound work (heavy computation); more overhead to start/communicate

ASYNCIO

- Single thread, cooperative multitasking via `async/await` and an event loop
- Excellent for many concurrent I/O waits (network calls) with low overhead

DEVELOPER PERSPECTIVE

- Ask first: is this CPU-bound or I/O-bound? That answer picks the model
- Don't reach for multiprocessing/asyncio until a simple synchronous script is provably too slow

threading vs multiprocessing vs asyncio

Approach	Code	Advantages	Disadvantages
threading	<code>threading.Thread(target=f).start()</code>	Simple API, good for I/O waits	Limited by the GIL for CPU-bound work
multiprocessing	<code>multiprocessing.Process(target=f).start()</code>	True parallelism, uses multiple cores	Higher memory/startup cost, harder data sharing
asyncio	<code>asyncio.run(main())</code>	Very low overhead for many I/O tasks	Requires async-aware libraries; one blocking call stalls everything

Basic Example

example.py

```
import threading
import time

def worker(name):
    time.sleep(1)
    print(f"{name} done")

t1 = threading.Thread(target=worker, args=("A",))
t2 = threading.Thread(target=worker, args=("B",))
t1.start(); t2.start()
t1.join(); t2.join()
print("all threads finished")
```

WHAT'S HAPPENING

- Thread(target=..., args=...) creates a thread that will run worker("A")
- start() launches the thread; join() blocks until it finishes
- Both sleeps overlap, so total wall time is ~1s, not ~2s

CONSOLE OUTPUT

```
A done
B done
all threads finished
```

Advanced / Developer Example

```
advanced_example.py

import asyncio

async def fetch(name, delay):
    await asyncio.sleep(delay)
    print(f"{name} finished after {delay}s")
    return name

async def main():
    results = await asyncio.gather(
        fetch("A", 1), fetch("B", 2)
    )
    print("results:", results)

asyncio.run(main())
```

WHAT'S HAPPENING

- `async def` defines a coroutine function; `await` suspends until it completes
- `asyncio.gather` runs multiple coroutines concurrently on one event loop
- `asyncio.sleep` is the async-friendly equivalent of `time.sleep`
- `asyncio.run()` is the standard entry point that starts and closes the event loop

CONSOLE OUTPUT

```
A finished after 1s
B finished after 2s
results: ['A', 'B']
```

input() vs sys.argv vs argparse

INPUT()

- Blocks and reads one line from stdin interactively — fine for demos/prompts
- Not suitable for scripts meant to be automated or piped into other tools

SYS.ARGV

- Raw list of strings passed on the command line; no parsing, validation, or help text

ARGPARSE

- Declarative: define expected arguments, types, defaults, and --help is generated for free
- Produces clear errors for missing/invalid arguments instead of silent IndexError

argparse Syntax

```
syntax.py

import argparse

parser = argparse.ArgumentParser(description="Resize an
image")
parser.add_argument("path", help="input file path")
parser.add_argument("--width", type=int, default=800)
parser.add_argument("--verbose", action="store_true")

args = parser.parse_args()
print(args.path, args.width, args.verbose)
```

HOW IT READS

- `add_argument("path")` defines a required positional argument
- `--width` is optional, typed as `int`, with a sensible default
- `action="store_true"` turns `--verbose` into a boolean flag (no value needed)
- `parser.parse_args()` reads `sys.argv` and produces a `Namespace` with typed fields
- Running the script with `-h` prints auto-generated usage/help text

Practical Example

```
practical_example.py

import argparse

parser = argparse.ArgumentParser(description="Summarize a CSV column")
parser.add_argument("csv_path")
parser.add_argument("--column", default="amount")
args = parser.parse_args()

import csv
total = 0.0
with open(args.csv_path, newline="") as f:
    for row in csv.DictReader(f):
        total += float(row[args.column])

print(f"Total {args.column}: {total:.2f}")
```

WHAT'S HAPPENING

- args.csv_path and args.column come from the command line, fully validated
- A missing csv_path argument produces a clear usage error, not a silent crash
- --column defaults to 'amount' but can be overridden per invocation
- This pattern turns a one-off script into a reusable CLI tool

CONSOLE OUTPUT

```
Total amount: 1284.50
```

Virtual Environments

WHAT & WHY

- A venv is an isolated Python install + site-packages directory per project
- Prevents dependency version conflicts between unrelated projects on the same machine
- Makes builds reproducible: requirements.txt pins exactly what a project needs

DEVELOPER PERSPECTIVE

- Create one venv per project, right in (or next to) the project directory
- Never install project dependencies into the system-wide Python — it becomes unmanageable

Creating & Activating a venv

- Create: `python -m venv .venv` (creates a `.venv` folder with an isolated interpreter)
- Activate (macOS/Linux): `source .venv/bin/activate`
- Activate (Windows, PowerShell): `.venv\Scripts\Activate.ps1`
- Activate (Windows, cmd.exe): `.venv\Scripts\activate.bat`
- Install deps inside it: `pip install -r requirements.txt`
- Deactivate anytime with: `deactivate`

Common Mistakes

✗ INCORRECT

```
pip install pandas # no venv active
```

✓ CORRECT

```
source .venv/bin/activate  
pip install pandas
```

✗ INCORRECT

```
name = sys.argv[1] # no argparse
```

✓ CORRECT

```
args = parser.parse_args()  
name = args.name
```

✗ INCORRECT

```
t = threading.Thread(target=cpu_heavy())
```

✓ CORRECT

```
t = threading.Thread(target=cpu_heavy)
```

✗ INCORRECT

```
await fetch() # outside an async function
```

✓ CORRECT

```
asyncio.run(fetch())
```

Common Mistakes — Why It Matters

- Mistake 1: Installing globally causes version conflicts across unrelated projects.
- Mistake 2: Raw `sys.argv` indexing crashes with `IndexError` on missing arguments; `argparse` fails gracefully.
- Mistake 3: Calling the function immediately (with `()`) runs it on the main thread instead of scheduling it.
- Mistake 4: `await` is only valid inside an `async def`; use `asyncio.run()` as the sync entry point.

Best Practices

MAINTAINABILITY

- Commit requirements.txt (or pyproject.toml), never the .venv folder itself
- Default to argparse over raw sys.argv for anything beyond a one-off throwaway script

PERFORMANCE

- Benchmark before parallelizing — concurrency adds real complexity for real gains only sometimes
- Prefer asyncio for many concurrent network calls; multiprocessing for heavy CPU work

Hands-on Exercise

PROBLEM STATEMENT

Build a CLI tool (argparse) that accepts a directory and a --workers count, then uses threading to check (simulated) 'availability' of N files concurrently, printing total elapsed time.

INPUT

```
python check.py ./data --workers 4
```

EXPECTED OUTPUT

```
Checked 10 files with 4 workers in 2.5s (vs ~10s sequential)
```

HINTS (no full solution here — give it a try first!)

- Each simulated check can just `time.sleep(1)` and print a result
- Split the file list into chunks, or launch one thread per file up to --workers at a time
- Compare a sequential run's `time.time()` delta against the threaded version

Automation in Practice

Automation scripts replace repetitive manual work — organizing files, chaining tools, calling APIs, or driving a browser.

THE PROBLEM IT SOLVES

Manual, repeated tasks (file cleanup, running tools in sequence, checking a site, polling an API) waste developer time and invite human error.

WHY IT MATTERS

A well-placed automation script often pays for itself the first week it exists.

WHERE IT'S USED

DevOps tooling, report generation, monitoring scripts, and QA/regression testing.

REAL-WORLD ANALOGY

"Automation is a small robot assistant that repeats a well-defined chore exactly, every time, without getting bored."

Categories of Automation

PROCESS AUTOMATION

- Local, filesystem-level tasks: organizing, renaming, cleaning up files
- Typical tools: os, shutil, pathlib

WORKFLOW AUTOMATION

- Chaining multiple tools/scripts into one repeatable pipeline
- Typical tools: subprocess, scheduling (cron / Task Scheduler / schedule library)

NETWORK & WEB AUTOMATION

- Network: calling REST APIs, polling services — typical tool: requests
- Web: driving an actual browser UI — typical tools: Selenium, Playwright

Basic Example

```
example.py

import os
import shutil

SORT_RULES = {".pdf": "documents", ".png": "images", ".jpg": "images"}

def organize(folder):
    for name in os.listdir(folder):
        ext = os.path.splitext(name)[1].lower()
        dest_folder = SORT_RULES.get(ext)
        if dest_folder:
            dest = os.path.join(folder, dest_folder)
            os.makedirs(dest, exist_ok=True)
            shutil.move(os.path.join(folder, name), dest)

organize("downloads")
```

WHAT'S HAPPENING

- `os.path.splitext` splits a filename into (base, extension)
- `SORT_RULES.get(ext)` looks up a destination folder, or `None` if the extension is unhandled
- `shutil.move` relocates the file once its destination directory exists
- This is process automation: a purely local file-organizing task

Practical Example

```
practical_example.py

import subprocess

def run_pipeline():
    steps = [
        ["python", "extract.py"],
        ["python", "transform.py"],
        ["python", "load.py"],
    ]
    for step in steps:
        result = subprocess.run(step, capture_output=True, text=True)
        if result.returncode != 0:
            print(f"Step {step} failed:\n{result.stderr}")
            return False
        print(f"Step {step} OK")
    return True
```

WHAT'S HAPPENING

- `subprocess.run()` launches another program and waits for it to finish
- `capture_output=True + text=True` gives back readable stdout/stderr strings
- Checking `returncode` lets the pipeline stop at the first failing step
- This is workflow automation: chaining independent scripts into one pipeline

CONSOLE OUTPUT

```
Step ['python', 'extract.py'] OK
Step ['python', 'transform.py'] OK
Step ['python', 'load.py'] OK
```

Advanced / Developer Example

```
advanced_example.py

import requests

def check_service_health(url):
    try:
        resp = requests.get(url, timeout=5)
        resp.raise_for_status()
        data = resp.json()
        return data.get("status") == "ok"
    except requests.RequestException as e:
        print(f"Health check failed: {e}")
        return False

if check_service_health("https://api.example.com/health"):
    print("Service healthy")
else:
    print("Service DOWN - alert on-call")
```

WHAT'S HAPPENING

- `requests.get(url, timeout=5)` issues an HTTP GET with a sane timeout
- `raise_for_status()` turns a 4xx/5xx response into a catchable exception
- `resp.json()` parses a JSON body straight into a Python dict
- This is network automation: polling a remote API on a schedule

CONSOLE OUTPUT

```
Service healthy
```

Web Automation (Browser-Level)

- Web automation drives an actual browser UI, not just an HTTP endpoint
- Use it when a task requires JavaScript rendering, login flows, or clicking through a UI
- Selenium and Playwright are the standard 3rd-party tools (not stdlib) for this
- Typical pattern: launch browser -> navigate(url) -> find_element(...) -> click()/type()
- Prefer requests/network automation whenever a plain API call can do the same job — it's faster and more stable
- Web automation is brittle to UI changes; keep selectors centralized and add explicit waits

Common Mistakes

✗ INCORRECT

```
subprocess.run("ls -la")
```

✓ CORRECT

```
subprocess.run(["ls", "-la"])
```

✗ INCORRECT

```
requests.get(url)
```

✓ CORRECT

```
requests.get(url, timeout=5)
```

✗ INCORRECT

```
shutil.move(src, dest) # dest dir missing
```

✓ CORRECT

```
os.makedirs(dest, exist_ok=True)  
shutil.move(src, dest)
```

✗ INCORRECT

```
resp.json() # without checking status
```

✓ CORRECT

```
resp.raise_for_status()  
resp.json()
```

Common Mistakes — Why It Matters

- Mistake 1: Passing a single string works only with `shell=True`; a list is the safe, portable form.
- Mistake 2: Without a timeout, a hung server can block the script indefinitely.
- Mistake 3: `shutil.move` fails if the destination directory doesn't already exist.
- Mistake 4: Parsing JSON from an error response often raises a confusing, unrelated exception.

Best Practices

MAINTAINABILITY

- Log every automated step's success/failure — silent automation is hard to debug later
- Make automation scripts idempotent: safe to re-run without duplicating work

PERFORMANCE

- Batch or rate-limit network/API calls instead of hammering a service in a tight loop

SECURITY

- Never hardcode API keys/credentials in an automation script — use environment variables

AI-Assisted Development

PROMPT

Here's a script that calls subprocess without checking for errors. Harden it for use in a scheduled automation job.

```
before_ai.py

import subprocess

def run(cmd):
    subprocess.run(cmd)

run(["python", "backup.py"])
```

AI ASSISTANT SUGGESTS

- Suggests checking returncode and raising/logging on failure
- Recommends capture_output=True to retain stderr for diagnostics
- Points out a missing timeout, which could hang a scheduled job indefinitely
- Suggests wrapping the call so a failure sends an alert rather than failing silently

DEVELOPER MUST STILL VERIFY

- Test the hardened script against a deliberately failing command
- Confirm logs/alerts actually fire in a real failure scenario, not just in theory
- Check that timeouts don't cut off a legitimately slow but successful job

Hands-on Exercise

PROBLEM STATEMENT

Write an automation script that: organizes a downloads folder by extension (process automation), then calls a public REST API and saves the JSON response to a timestamped file (network automation).

INPUT

A downloads/ folder with mixed file types; a public JSON API URL

EXPECTED OUTPUT

Organized 12 files into 3 folders.
Saved API response to snapshot_2026-09-13T10-00-00.json

HINTS (no full solution here — give it a try first!)

- Reuse the extension-sorting pattern from the Basic Example
- Use `datetime.now().strftime(...)` to build the timestamped filename
- Wrap the API call in `try/except requests.RequestException` so one failure doesn't crash the whole script

AI Tools for Developers

AI coding assistants can optimize, generate, and help debug code, and act as a pair-programming partner throughout the day.

THE PROBLEM IT SOLVES

Developers spend significant time on boilerplate, performance tuning, and tracking down bugs — all tasks AI can accelerate.

WHY IT MATTERS

Used well, AI tools compress hours of work into minutes; used carelessly, they introduce unverified assumptions into production.

WHERE IT'S USED

IDE assistants (Copilot, etc.), chat-based coding assistants, and CI-integrated review bots.

REAL-WORLD ANALOGY

"An AI coding assistant is a fast, well-read junior partner — useful, but everything it produces still needs your sign-off."

Four Ways Developers Use AI

CODE OPTIMIZATION

- Ask AI to identify inefficient algorithms/data structures and propose faster alternatives

CODE GENERATION

- Generate boilerplate, scaffolding, and first-draft implementations from a clear spec

BUG DETECTION & RESOLUTION

- Paste a stack trace or failing test; ask AI to hypothesize root causes and a fix

PAIR PROGRAMMING

- In-editor tools (e.g. GitHub Copilot) suggest completions as you type, in real time

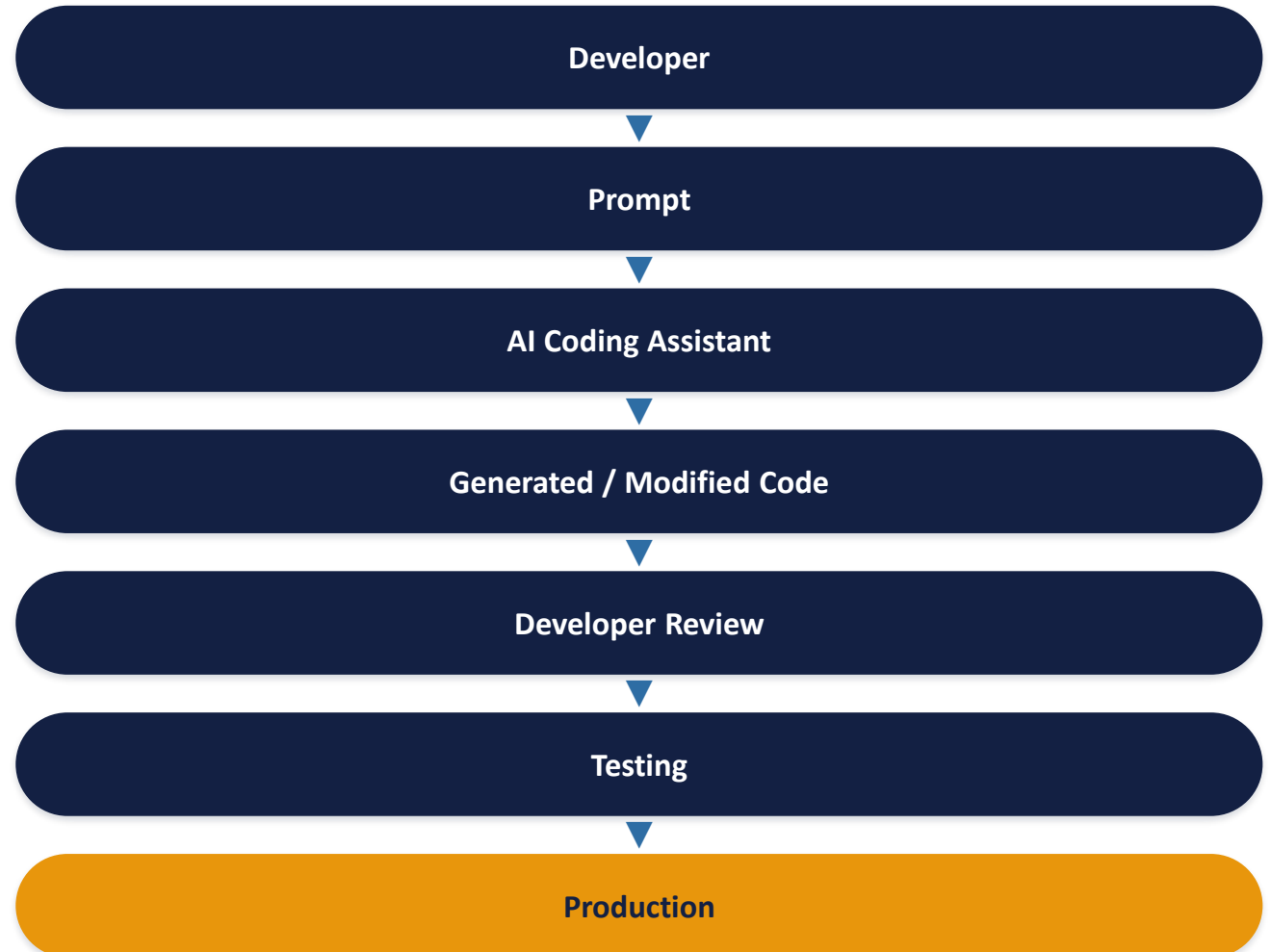
Code Optimization with AI

WHAT IS IT

Give an AI assistant a working-but-slow function and ask it to identify the complexity problem and propose a faster equivalent.

WHY DEVELOPERS NEED IT

Manually spotting $O(n^2)$ patterns in a large codebase is slow; an AI assistant can flag them quickly, but the fix must still be verified.



Code Optimization with AI — In Practice

EXAMPLE PROMPT

This function finds duplicate values in a list. It's slow on large inputs. Identify its time complexity and rewrite it to be faster, explaining the trade-off.

```
before.py

def find_duplicates(items):
    duplicates = []
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j] and items[i] not
in duplicates:
                duplicates.append(items[i])
    return duplicates
```

```
after_ai.py

def find_duplicates(items):
    seen = set()
    duplicates = set()
    for item in items:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)
    return list(duplicates)
```

Code Optimization with AI — Review & Practice

BEFORE YOU TRUST THE OUTPUT — REVIEW FOR:

Correctness

Performance

Edge cases

Maintainability

HANDS-ON ACTIVITY

Run both versions on a 10,000-item list with `time.time()` and compare wall-clock time directly.

Practical Example

```
practical_example.py

import time

data = list(range(5000)) + list(range(2500))  # forces duplicates

start = time.time()
find_duplicates_naive(data)
print("naive:", time.time() - start)

start = time.time()
find_duplicates(data)  # set-based version
print("optimized:", time.time() - start)
```

WHAT'S HAPPENING

- Benchmarking both versions on the same input is how you validate an AI's optimization claim
- The nested-loop version scales $O(n^2)$; the set-based version scales $O(n)$
- `time.time()` deltas give a simple, reproducible before/after comparison
- Never accept 'this is faster' from an AI response without measuring it yourself

CONSOLE OUTPUT

```
naive: 1.842
optimized: 0.0015
```

Code Generation & Bug Detection with AI

CODE GENERATION

- Best for well-specified, boilerplate-heavy tasks: CRUD scaffolding, parsers, test stubs
- Weakest on business-specific logic the assistant has no context for

BUG DETECTION & RESOLUTION

- Paste the traceback plus the minimal relevant code, not the whole file
- Ask for the root cause explanation, not just a patch — helps you evaluate the fix

PAIR PROGRAMMING (COPILOT-STYLE)

- Inline suggestions work best with descriptive names, docstrings, and nearby examples
- Treat every accepted suggestion as a first draft, not a finished implementation

Common Mistakes

✗ INCORRECT

Accepted AI fix, shipped without running tests

✓ CORRECT

Ran full test suite + new edge-case tests before merging

✗ INCORRECT

"AI says this is $O(n)$, must be faster"

✓ CORRECT

Benchmarked both versions on realistic data sizes

✗ INCORRECT

Pasted an entire 2000-line file as context

✓ CORRECT

Pasted the failing function + relevant imports/tests

✗ INCORRECT

Accepted a Copilot suggestion with an unfamiliar API call

✓ CORRECT

Looked up the API call's docs before accepting

Common Mistakes — Why It Matters

- Mistake 1: AI-suggested fixes can be plausible-looking but subtly wrong or incomplete.
- Mistake 2: AI complexity/performance claims must be benchmarked, never assumed.
- Mistake 3: Excess irrelevant context dilutes the assistant's focus and wastes context budget.
- Mistake 4: Assistants sometimes hallucinate plausible-looking but nonexistent function signatures.

Best Practices

TRUSTING AI-OPTIMIZED CODE

- Performance claims must be benchmarked on realistic data — never assumed from the explanation alone
- Re-derive or verify the stated Big-O complexity yourself for anything performance-critical

MAINTAINABILITY

- Keep the original version in version control/comments until the optimized version is proven

SECURITY

- Scan AI-generated code for hardcoded secrets, unsafe eval/exec, or unvalidated input handling

Hands-on Exercise

PROBLEM STATEMENT

Take a naive function that checks if a list has any duplicate values (nested loop). Ask an AI assistant to optimize it, then benchmark the naive vs. optimized version on a 20,000-item list and report the speedup factor.

INPUT

A list of 20,000 integers with some duplicates

EXPECTED OUTPUT

Naive: 6.7s, Optimized: 0.004s, Speedup: ~1675x

HINTS (no full solution here — give it a try first!)

- Use `time.time()` before/after each version, on the exact same input list
- A set-based membership check is the typical $O(n)$ rewrite
- Report the speedup as a ratio, not just 'it was faster'

GitHub Copilot, Context Engineering & Adoption

Context engineering shapes what an AI assistant 'sees' to improve its output; Copilot applies this inline in the editor; adoption strategy scales it across a team responsibly.

THE PROBLEM IT SOLVES

Poor context produces generic, often-wrong suggestions; ad-hoc team adoption of AI tools creates inconsistent quality and governance gaps.

WHY IT MATTERS

The gap between a mediocre and a great AI-assisted developer is mostly about context and process, not the model itself.

WHERE IT'S USED

Daily IDE work with Copilot/assistants, and organization-wide AI tooling rollouts.

REAL-WORLD ANALOGY

"Context engineering is like briefing a new contractor thoroughly before they start — the better the brief, the better the work."

Context Engineering: Prompting Techniques

WHAT MORE CONTEXT CHANGES

- Open/relevant files in the editor sharpen suggestions to your actual codebase conventions
- Descriptive names, docstrings, and type hints act as free context for the assistant
- Examples of desired input/output steer generation more reliably than descriptions alone

TECHNIQUES

- State the goal, constraints, and target style/framework explicitly
- Break large tasks into smaller, well-scoped prompts rather than one giant request
- Reference existing patterns in the repo ("match the style of parser.py")

Context Engineering: Weak to Expert

WEAK PROMPT

Write a function to process orders.

BETTER PROMPT

Write a Python function that reads orders from a CSV (id, amount, status), filters status == 'paid', and returns the total amount.

The expert prompt supplies exact shape, error handling expectations, and style constraints, so the assistant's first draft needs far less rework.

Context Engineering: Weak to Expert — Expert-Level Prompt

EXPERT PROMPT

Act as a senior Python developer working in our existing codebase style (see `orders.py` for conventions: type hints, docstrings, no bare except).

Write a function `total_paid_amount(csv_path: str) -> float` that:

1. Reads orders from a CSV with columns `id`, `amount`, `status`.
 2. Filters to rows where `status == 'paid'`.
 3. Returns the sum of `amount` as a float, or `0.0` if the file is empty.
 4. Raises a clear `ValueError` if the `'amount'` or `'status'` column is missing.
- Include a short docstring and one usage example in a comment.

Vibe Coding: Matching Tone & Intent

WHAT IT MEANS

- Aligning AI output with the intended audience and purpose, not just technical correctness
- A prototype for a demo can be quick-and-dirty; production code for a regulated system cannot

IN PRACTICE

- Tell the assistant the audience/stakes: "internal quick script" vs. "customer-facing production API"
- Ask explicitly for the level of error handling, comments, and test coverage the context demands

DEVELOPER PERSPECTIVE

- Vibe coding speeds up exploration — but the 'vibe' output still needs a deliberate hardening pass before shipping

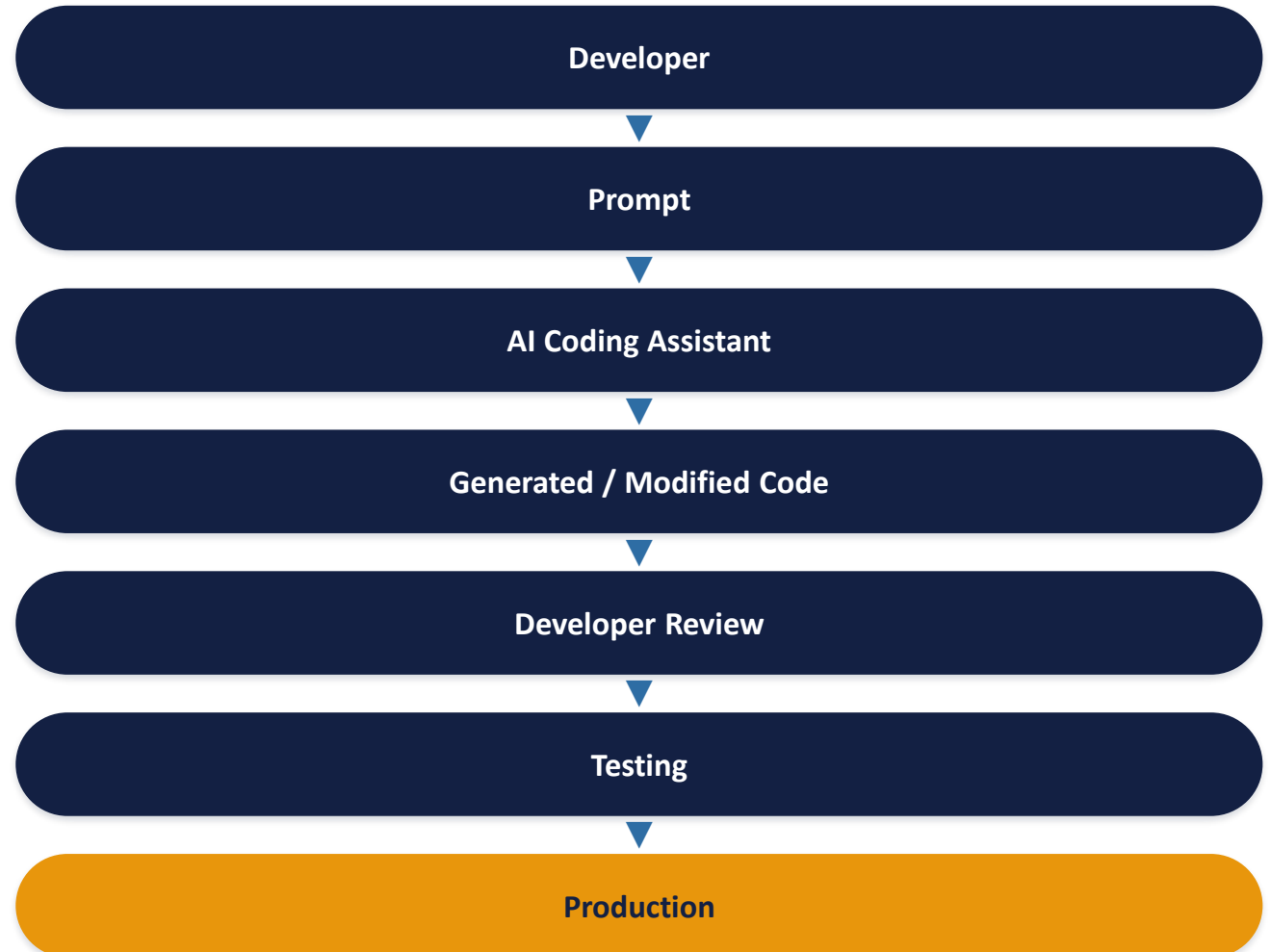
GitHub Copilot in Practice

WHAT IS IT

GitHub Copilot suggests code completions inline, in real time, based on the current file, open tabs, and comments/docstrings as context.

WHY DEVELOPERS NEED IT

It accelerates prototyping and eliminates repetitive boilerplate, letting developers focus on logic and design decisions.



GitHub Copilot in Practice — In Practice

EXAMPLE PROMPT

Parse a JSON API response and return a list of active user emails, skipping any user missing an 'email' field

```
before.py

def active_user_emails(response_json):
    # Copilot completes from here based on the
    # comment above
```

```
after_ai.py

def active_user_emails(response_json):
    emails = []
    for user in response_json.get("users", []):
        if user.get("active") and "email" in
user:
            emails.append(user["email"])
    return emails
```

GitHub Copilot in Practice — Review & Practice

BEFORE YOU TRUST THE OUTPUT — REVIEW FOR:

Correctness

Edge cases

Security

Maintainability

HANDS-ON ACTIVITY

Write only a docstring/comment for a small utility function, accept Copilot's suggestion, then review it line by line for correctness.